DCIA PoC — 3–4 week compressed task checklist

Source: MDL PS-1 & PS-2 · Derived from PoC Instructions v1.0

Day 1 — team alignment (2 hours, all three engineers)

- [ ] Agree API contract: copy the `/api/chat` request/response JSON shape from the PoC instructions doc and sign off — frontend builds against this from day one
- [ ] Confirm LLM provider and get API key provisioned (OpenAI GPT-4o recommended for speed)
- [ ] Confirm Neo4j approach: Docker locally or Neo4j Aura Free Tier (Aura avoids Docker setup time)
- [ ] Set up shared Git repo with `backend/`, `frontend/`, `graph-seed/` folders
- [ ] Each engineer pastes their `.env` template — no keys in git

---

Week 1 — foundations (parallel workstreams)

Graph engineer

- [ ] Day 1–2: Docker Compose up, Neo4j accessible at `localhost:7474`
- [ ] Day 2–3: Run all CREATE node Cypher from Step 1b (7 systems, 5 components, 3 historical changes, 4 class rules)
- [ ] Day 3–4: Run all CREATE relationship Cypher from Step 1c
- [ ] Day 4–5: Run verification queries from Step 1d — confirm cascade traversal returns ⩾3 hops
- [ ] Day 5: Share Neo4j credentials with backend engineer; confirm `bolt://localhost:7687` reachable

Backend engineer

- [ ] Day 1–2: FastAPI project scaffold, `requirements.txt` install, health endpoint working at `localhost:8000/health`
- [ ] Day 2–3: `neo4j_service.py` — `get_cascade_impacts()` function, tested with hardcoded `SYS-001`
- [ ] Day 3–4: `llm_service.py` — `extract_intent()` function, tested with a sample propulsion prompt

- [ ] Day 4–5: `cost_model.py` — parametric stub returning P10/P50/P90 for a given discipline + magnitude
- [ ] Day 5: All three services individually unit-tested; backend engineer and graph engineer do a 30-min joint smoke test

Frontend engineer

- [ ] Day 1: `create-react-app dcia-ui --template typescript`, Tailwind + Recharts installed
- [ ] Day 2–3: Build the chat input panel and sample prompt buttons — wired to a **local mock JSON file** (not the API yet)
- [ ] Day 4–5: Build the impact table component (discipline, severity, hops, reason) rendering from mock data
- [ ] Day 5: UI renders all mock data without errors; ready to swap mock for live API

Week 2 — core integration

Backend engineer

- [ ] Day 1–2: Wire `main.py` `/api/chat` endpoint — calls all three services in sequence, returns unified JSON
- [ ] Day 2: Share Postman/curl example with frontend engineer confirming live endpoint works for Scenario 1
- [ ] Day 3: Add `get_class_rules_for_systems()` and `get_similar_historical_changes()` to the endpoint
- [ ] Day 4–5: Run test Scenario 1 (propulsion +15%) and Scenario 2 (generator replacement) end-to-end; fix any graph query issues with graph engineer

Frontend engineer

- [ ] Day 1–2: Swap mock data for live `axios.post('/api/chat')` call — confirm chat round-trip works
- [ ] Day 3: Build class rules panel (amber left-border cards with society + clause)
- [ ] Day 4: Build historical analogues section (change ID, description, actual cost, delay days)
- [ ] Day 5: All four UI sections rendering live data; demo to backend engineer

Graph engineer

- [ ] Day 1–3: Fix any schema issues surfaced during backend integration
- [ ] Day 4–5: Add CORS to FastAPI if needed; help backend engineer debug any Cypher query edge cases

---

Week 3 — test scenarios + polish

All engineers (joint)

- [ ] Day 1: Run Scenario 1 — propulsion +15%. Confirm ⩾3 impacted systems, ⩾2 class rules, historical analogue HC-2022-014 returned
- [ ] Day 2: Run Scenario 2 — generator replacement. Confirm electrical + weapons cascade, CR-DNV-002 flagged
- [ ] Day 3: Run Scenario 3 — structural stiffening. Confirm structure → outfitting LOW severity cascade
- [ ] Day 4: Run Scenario 4 — fuel capacity +20%. Confirm lower cost estimate than propulsion scenarios
- [ ] Day 5: Fix blockers from scenarios; freeze code

Frontend engineer (polish, if time permits)

- [ ] Add cost bar chart (Recharts BarChart with P10/P50/P90) — this was deferred but can be added in ~3 hours using the code in the PoC doc
- [ ] Add severity colour badges (red/amber/green) to the impact table

---

Week 4 — buffer and demo prep (use only if needed)

- [ ] Remaining bug fixes from Week 3 scenario runs
- [ ] Prepare demo script: walk through Scenario 1 live, show graph in Neo4j Browser side-by-side
- [ ] Prepare 3 slides: problem (from PS-1/PS-2), demo screenshot, what comes next (RAG, cost model training, PLM integration)
- [ ] Acceptance criteria sign-off: run through all 8 criteria from the PoC instructions doc

---

What is explicitly NOT in scope for this timeline

Deferred item	Why	When to add
RAG / vector similarity for historical changes	Requires embedding pipeline setup (+3 days)	Phase 2, post-PoC

Deferred item	Why	When to add
Recharts cost bar chart	Nice-to-have polish	Week 3 if time permits
BOM sync / 3D model delta viewer	Requires CAD parser (+1 week)	Phase 2
Auth / RBAC	Not needed for internal demo	Before customer pilot
PLM / ERP integration	Requires MDL system access	Phase 2
Production Neo4j cluster	Local Docker sufficient for PoC	Before go-live

Risk register

Risk	Likelihood	Impact	Mitigation
LLM intent extraction returns wrong `system_id`	Medium	High — breaks graph traversal	Add hardcoded fallback map in `extract_intent()`; test all 4 scenario prompts on day 1
Neo4j Bolt port blocked by corporate firewall	Low	High	Use Neo4j Aura Free Tier (cloud-hosted) as alternative from day 1
Frontend can't reach FastAPI (CORS)	Medium	Medium	Add `CORSMiddleware(allow_origins=["*"])` on day 1 — already in the code
Week 2 API handoff slips	Medium	High	Frontend engineer builds complete mock UI in Week 1 so they're never blocked
OpenAI rate limits during testing	Low	Low	Cache last response in memory; add `time.sleep(1)` between test scenario runs

Checklist version 1.0 · 3–4 week compressed plan · References: MDL PS-1, PS-2 · PoC Instructions v1.0